

Basic syntax overview

June 17, 2026 2:43 PM

<https://kotlinlang.org/docs/basic-syntax.html>

Basic syntax overview	
Package definition and imports	
Program entry point	
Print to the standard output	
Read from the standard input	
Functions	
Variables	
Creating classes and instances	
Comments	
String templates	
Conditional expressions	
for loop	
while loop	
when expression	
Ranges	
Collections	
Nullable values and null checks	
Type checks and automatic casts	

Basic syntax overview

[Edit page](#) Last modified: 01 April 2026

This is a collection of basic syntax elements with examples. At the end of every section, you'll find a link to a detailed description of the related topic.

You can also learn all the Kotlin essentials with the free Kotlin Core track [↗](#) by JetBrains Academy.

Package definition and imports

Package specification should be at the **top** of the source file:

```
package my.demo

import kotlin.text.*

// ...
```

It is not required to match directories and packages: **source files** can be **placed arbitrarily** in the file system.

See Packages.

Program entry point

An entry point of a Kotlin application is the **main** function:

```
fun main() {
    println("Hello world!")
}
```

[Open in Playground](#) →

Target: JVM Running on v24.0

Another form of **main** accepts a variable number of **String** arguments:

```
fun main(args: Array<String>) {
    println(args.contentToString())
}
```

```
fun main(args: Array<String>) {  
    println(args.contentToString())  
}
```

[Open in Playground](#) →

Target: JVM Running on v2.4.0

Print to the standard output

`print` prints its argument to the standard output:

```
print("Hello ")  
print("world!")
```

[Open in Playground](#) →

Target: JVM Running on v2.4.0

`println` prints its arguments and adds a line break, so that the next thing you print appears on the next line:

```
println("Hello world!")  
println(42)
```

[Open in Playground](#) →

Target: JVM Running on v2.4.0

Read from the standard input

The `readln()` function reads from the standard input. This function reads the entire line the user enters as a string.

You can use the `println()`, `readln()`, and `print()` functions together to print messages requesting and showing user input:

```
// Prints a message to request input  
println("Enter any word: ")  
  
// Reads and stores the user input. For example: Happiness  
val yourWord = readln()  
  
// Prints a message with the input  
print("You entered the word: ")  
print(yourWord)  
// You entered the word: Happiness
```

For more information, see [Read standard input](#).

Functions

A function with two `Int` parameters and `Int` return type:

```
fun sum(a: Int, b: Int): Int {  
    return a + b  
}
```

Open in Playground → Target: JVM Running on v2.4.0

A function body can be an `expression`. Its `return type` is `inferred`:

```
fun sum(a: Int, b: Int) = a + b
```

Open in Playground → Target: JVM Running on v2.4.0

A function that `returns no meaningful` value:

```
fun printSum(a: Int, b: Int): Unit {  
    println("sum of $a and $b is ${a + b}")  
}
```

Open in Playground → Target: JVM Running on v2.4.0

`Unit` return type can be omitted:

```
fun printSum(a: Int, b: Int) {  
    println("sum of $a and $b is ${a + b}")  
}
```

Open in Playground → Target: JVM Running on v2.4.0

See [Functions](#).

Variables

In Kotlin, you declare a variable starting with a keyword, `val` or `var`, followed by the `name` of the variable.

Use the `val` keyword to declare variables that are `assigned` a value `only once`. These are `immutable`, read-only local variables that can't be reassigned a different value after initialization:

```
// Declares the variable x and initializes it with the value of 5  
val x: Int = 5  
// 5
```

Open in Playground → Target: JVM Running on v2.4.0

Use the `var` keyword to declare variables that `can be reassigned`. These are mutable variables, and you can change their values after initialization:

```
// Declares the variable x and initializes it with the value of 5  
var x: Int = 5  
// Reassigns a new value of 6 to the variable x  
x += 1  
// 6
```

Open in Playground → Target: JVM Running on v2.4.0

Kotlin supports `type inference` and automatically identifies the `data type` of a declared variable. When declaring a variable, you can omit the type after the variable name:

```
// Declares the variable x with the value of 5; 'Int' type is inferred  
val x = 5  
// 5
```

Open in Playground → Target: JVM Running on v2.4.0

You can `use variables` only `after initializing` them. You can either initialize a variable at the moment of declaration or declare a variable first and initialize it later. In the second case, you must specify the data type:

```
// Initializes the variable x at the moment of declaration; type is not required  
val x = 5  
// Declares the variable c without initialization; type is required  
val c: Int  
// Initializes the variable c after declaration  
c = 3  
// 5
```

```
// Declares the variable c without initialization; type is required
val c: Int
// Initializes the variable c after declaration
c = 3
// 5
// 3
```

Open in Playground →

Target: JVM Running on v2.4.0

You can **declare** variables at the **top level**:

```
val PI = 3.14
var x = 0

fun incrementX() {
    x += 1
}

// x = 0; PI = 3.14
// incrementX()
// x = 1; PI = 3.14
```

Open in Playground →

Target: JVM Running on v2.4.0

For information about declaring properties, see [Properties](#).

Creating classes and instances

To define a class, use the **class** keyword:

```
class Shape
```

Properties of a class can be listed in its declaration or body:

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
```

The default **constructor** with parameters listed in the class declaration is available automatically:

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}

fun main() {
    val rectangle = Rectangle(5.0, 2.0)
    println("The perimeter is ${rectangle.perimeter}")
}
```

Open in Playground →

Target: JVM Running on v2.4.0

Inheritance between classes is declared by a colon **:**. Classes are **final** by default; to make a class inheritable, mark it as **open**:

```
open class Shape
```

```
class Rectangle(val height: Double, val length: Double): Shape() {
    val perimeter = (height + length) * 2
}
```

For more information about constructors and inheritance, see [Classes and Objects and instances](#).

'open' to make class inheritable

Comments

Just like most modern languages, Kotlin supports **single-line** (or **end-of-line**) and **multi-line** (**block**) comments:

```
// This is an end-of-line comment

/* This is a block comment
   on multiple lines. */
```

Block comments in Kotlin can be nested:

```
/* The comment starts here
   /* contains a nested comment */
   and ends here. */
```

See [Documenting Kotlin Code](#) for information on the documentation comment syntax.

String templates

```
var a = 1
// simple name in template:
val s1 = "a is $a"

a = 2
// arbitrary expression in template:
val s2 = "${s1.replace("is", "was")}, but now is $a"
```

Open in Playground → Target: JVM Running on v 2.4.0

See [String templates](#) for details.

Conditional expressions

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) {
        return a
    } else {
        return b
    }
}
```

Open in Playground → Target: JVM Running on v 2.4.0

In Kotlin, `if` can also be used as an expression:

```
fun maxOf(a: Int, b: Int) = if (a > b) a else b
```

Open in Playground → Target: JVM Running on v 2.4.0

See [if -expressions](#).

for loop

```
val items = listOf("apple", "banana", "kiwifruit")
for (item in items) {
    println(item)
}
```

Open in Playground → Target: JVM Running on v2.4.0

Or:

```
val items = listOf("apple", "banana", "kiwifruit")
for (index in items.indices) {
    println("item at $index is ${items[index]}")
}
```

Open in Playground → Target: JVM Running on v2.4.0

See for loop.

while loop

```
val items = listOf("apple", "banana", "kiwifruit")
var index = 0
while (index < items.size) {
    println("item at $index is ${items[index]}")
    index++
}
```

Open in Playground → Target: JVM Running on v2.4.0

See while loop.

when expression

```
fun describe(obj: Any): String =
    when (obj) {
        1 -> "One"
        "Hello" -> "Greeting"
        is Long -> "Long"
        !is String -> "Not a string"
        else -> "Unknown"
    }
```

Open in Playground → Target: JVM Running on v2.4.0

See when expressions and statements.

Ranges

Check if a number is within a range using `in` operator:

```
val x = 10
val y = 9
if (x in 1..y+1) {
    println("fits in range")
}
```

[Open in Playground](#) →

Target: JVM Running on v2.4.0

Check if a number is out of range:

```
val list = listOf("a", "b", "c")

if (-1 in 0..list.lastIndex) {
    println("-1 is out of range")
}
if (list.size !in list.indices) {
    println("list size is out of valid list indices range, too")
}
```

[Open in Playground](#) →

Target: JVM Running on v2.4.0

Iterate over a range:

```
for (x in 1..5) {
    print(x)
}
```

[Open in Playground](#) →

Target: JVM Running on v2.4.0

Or over a progression:

```
for (x in 1..10 step 2) {
    print(x)
}
println()
for (x in 9 downTo 0 step 3) {
    print(x)
}
```

[Open in Playground](#) →

Target: JVM Running on v2.4.0

See [Ranges](#) and [progressions](#).

Collections

Iterate over a collection:

```
for (item in items) {  
    println(item)  
}
```

[Open in Playground](#)

Target: JVM Running on v2.4.0

Check if a collection contains an object using `in` operator:

```
when {  
    "orange" in items -> println("juicy")  
    "apple" in items -> println("apple is fine too")  
}
```

[Open in Playground](#)

Target: JVM Running on v2.4.0

Use lambda expressions to filter and map collections:

```
val fruits = listOf("banana", "avocado", "apple", "kiwifruit")  
fruits  
    .filter { it.startsWith("a") }  
    .sortedBy { it }  
    .map { it.uppercase() }  
    .forEach { println(it) }
```

[Open in Playground](#)

Target: JVM Running on v2.4.0

See Collections overview.

Nullable values and null checks

A reference must be explicitly marked as nullable when a `null` value is possible. Nullable type names have `?` at the end. For example, `Int?`.

Return `null` if `str` does not hold an integer:

```
fun parseInt(str: String): Int? {  
    return str.toIntOrNull()  
}
```

Use a function returning nullable value:

```
fun printProduct(arg1: String, arg2: String) {  
    val x = parseInt(arg1)  
    val y = parseInt(arg2)  
  
    // Using `x * y` yields error because they may hold nulls.  
    if (x != null && y != null) {  
        // x and y are automatically cast to non-nullable after null check  
        println(x * y)  
    }  
    else {  
        println("'${arg1}' or '${arg2}' is not a number")  
    }  
}
```

[Open in Playground](#)

Target: JVM Running on v2.4.0

or:

```
// ...  
if (x == null) {  
    println("Wrong number format in arg1: '$arg1'")  
    return  
}  
if (y == null) {  
    println("Wrong number format in arg2: '$arg2'")  
    return  
}  
  
// x and y are automatically cast to non-nullable after null check  
println(x * y)
```

[Open in Playground](#)

Target: JVM Running on v2.4.0

See Null-safety.

See [Null-safety](#).

Type checks and automatic casts

The `is` operator checks if an expression is an instance of a type. If an immutable local variable or property is checked for a specific type, there's no need to cast it explicitly:

```
fun getStringLength(obj: Any): Int? {  
    if (obj is String) {  
        // `obj` is automatically cast to `String` in this branch  
        return obj.length  
    }  
  
    // `obj` is still of type `Any` outside of the type-checked branch  
    return null  
}
```

Open in Playground →

Target: JVM Running on v2.4.0

or:

```
fun getStringLength(obj: Any): Int? {  
    if (obj !is String) return null  
  
    // `obj` is automatically cast to `String` in this branch  
    return obj.length  
}
```

Open in Playground →

Target: JVM Running on v2.4.0

or even:

```
fun getStringLength(obj: Any): Int? {  
    // `obj` is automatically cast to `String` on the right-hand side of `&&`  
    if (obj is String && obj.length >= 0) {  
        return obj.length  
    }  
  
    return null  
}
```

Open in Playground →

Target: JVM Running on v2.4.0

See [Classes and Type casts](#).